Customer Churn Prediction

Project Summary: from Problem Statement to Sprints

Detail	Value
Course	Software Engineering Mini Project, PES University
Team	Hackforge
Members (SRN)	Akshaya B (PES1UG24AM026); Ashwin C H (PES1UG24AM900); Amar Patil (PES1UG24AM031)
Repository	github.com/Akshayshetty742/hackforge-customer-churn

1. Problem Statement

Telecom and subscription companies lose revenue when customers cancel their service (churn). Winning a new customer costs five to seven times more than keeping an existing one, so companies need to know which customers are likely to leave before they do.

The goal of this project is to build a system that predicts whether a customer will churn from their profile (demographics, services, contract and billing), explains the main risk factors, and helps a retention team decide what to do. A key difficulty is class imbalance: only about one in four customers churns, so a model can look accurate while missing most churners.

2. Objectives

- Predict customer churn with high recall, so that most real churners are caught.
- Handle class imbalance without throwing away data.
- Provide a simple web application for single and batch predictions, explanations and what-if analysis.
- Compare classical machine learning, deep learning and quantum machine learning on the same problem.
- Follow software engineering practice: requirements, design, testing, version control, CI and sprints.

3. Dataset

Item	Description
Source	IBM Telco Customer Churn sample dataset
Size	7,043 customers, 19 input features and the Churn target
Features	Demographics (gender, senior citizen, partner, dependents), services (phone, internet and add-ons), contract, billing method, tenure, monthly and total charges
Target	Churn (Yes / No); 26.5% of customers churn
Split	Stratified 80% training / 20% test (1,409 customers), random seed 42

4. Requirements

The requirements are specified in the SRS (IEEE 830): 24 functional, 13 non-functional, 7 interface and 10 security requirements and objectives. The main ones are:

ID	Requirement
FR-01, FR-02	Load the dataset and stop with a clear error if it is missing or invalid.
FR-05, FR-10	Train an XGBoost classifier with class weighting.
FR-15, FR-16	Web form with 19 labelled fields and sample customers; every input is validated.
FR-17	Show the predicted class, churn probability and risk tier (Low, Moderate, High).
FR-18, FR-19	Evaluate the model with an adjustable threshold; score a CSV batch of customers.
FR-20	Explain risk drivers and simulate retention actions (what-if).
FR-21 to FR-24	Deep learning and quantum models, benchmarked and shown side by side with XGBoost.
NFR-01, NFR-11	Training under 300 seconds; one prediction under 200 ms.
NFR-13	pylint score of at least 8.5 and at least 85% test coverage.
SR-01, SR-04	No personal identifiers as model inputs; all processing stays local.

5. System Design

The system uses a layered architecture with a pipeline for training:

Layer	Components	Role
Presentation	app.py (Streamlit)	Web application with four pages: prediction, batch scoring, model evaluation, about
Service	prediction_pipeline.py	Validation, preprocessing, prediction, evaluation, batch scoring, explanation
Models	classical_models.py, dml_model.py, qml_model.py	XGBoost and Logistic Regression, PyTorch MLP, PennyLane quantum classifier
Training	training_common.py, 01 and 02 scripts	Shared data split and metrics; production training; three-paradigm benchmark
Storage	Model files, dataset, MLflow store	Saved models and encoders, DVC-tracked data, experiment records

Technology: Python, pandas, scikit-learn, XGBoost, PyTorch, PennyLane, Streamlit, DVC, MLflow, pytest, GitHub Actions.

6. Methodology

- Preprocessing: remove the customer ID, convert blank total charges to zero, label-encode text columns.
- Class imbalance: class weighting (scale_pos_weight of about 2.77) instead of oversampling or deleting data.
- Classical model: XGBoost tuned with GridSearchCV and stratified 3-fold cross validation.
- Deep learning: a multi-layer perceptron (64 and 32 units, batch normalisation, dropout, early stopping).
- Quantum ML: a 6-qubit variational quantum classifier with 3 entangling layers, simulated with PennyLane.
- Evaluation: accuracy, precision, recall, F1 and ROC-AUC on the same held-out test set for every model.

7. Results

Production model (XGBoost) on the 1,409-customer test set:

Accuracy	Precision (churn)	Recall (churn)	F1 (churn)	ROC-AUC
75.7%	52.9%	75.1%	0.621	0.841

Comparison of the three paradigms on the same split:

Paradigm	Model	Accuracy	Recall (churn)	ROC-AUC	Parameters
ML	XGBoost	75.7%	79.4%	0.846	5,628
DML	Deep MLP	73.2%	79.7%	0.841	5,249
ML	Logistic Regression	73.7%	79.4%	0.839	20
QML	Quantum VQC	73.7%	75.4%	0.829	61

All three paradigms reach similar quality. The deep network has the highest recall, and the quantum model comes within 0.02 ROC-AUC of XGBoost with only 61 trainable parameters.

8. Application

- Customer Churn Prediction: form with sample customers, prediction, risk gauge, risk drivers, what-if simulator, and ML / DML / QML predictions side by side.
- Batch Customer Scoring: score a CSV file and download a risk-sorted list.
- Model Evaluation: saved and live metrics, confusion matrix, adjustable decision threshold.
- About the Project: problem, approach and sprint summary.

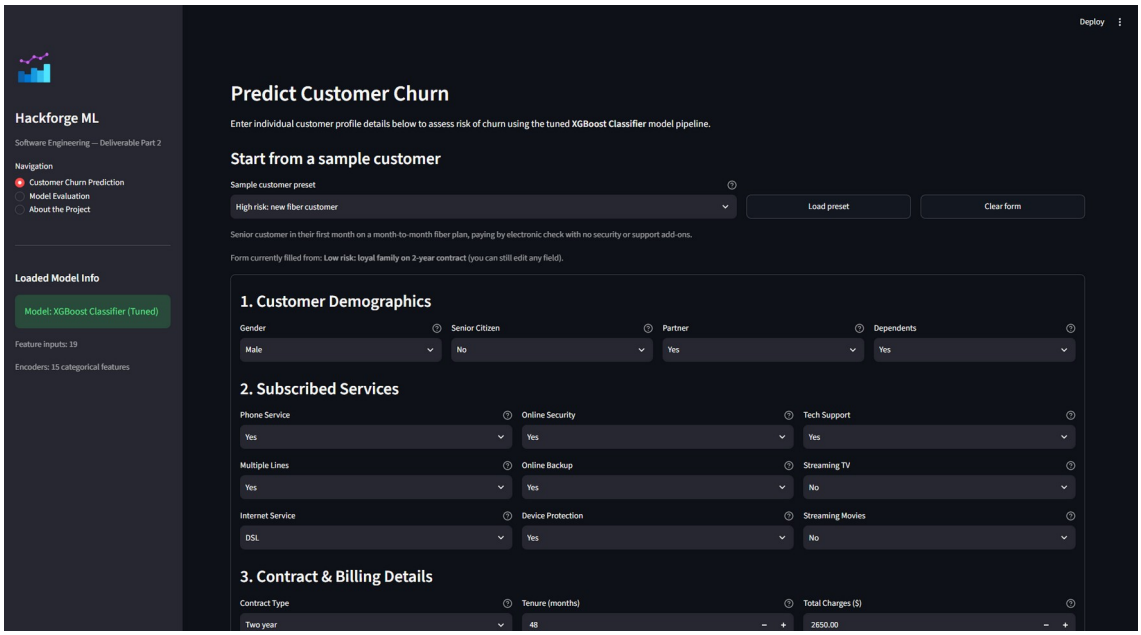


Figure 1. Prediction page of the web application

9. Testing and Quality

Measure	Result
Automated tests	152 of 152 passed
Test Plan test cases	26 of 26 passed (functional, non-functional and security)
Requirements validated	54 of 54
Statement coverage	89.1%
Branch coverage	79.4%
pylint score	9.82 / 10
Training time / prediction time	15.3 s / 11.3 ms

Tests run automatically on every pull request with GitHub Actions. Experiments are reproducible with a DVC pipeline, and every training run is recorded in MLflow.

10. Sprints

Sprint	Window	Stories	Story points completed	Completion
Sprint 1	19 to 23 October 2026	5	23	100%
Sprint 2	26 to 30 October 2026	6	26	100%
Sprint 3	2 to 6 November 2026	10	44	100%

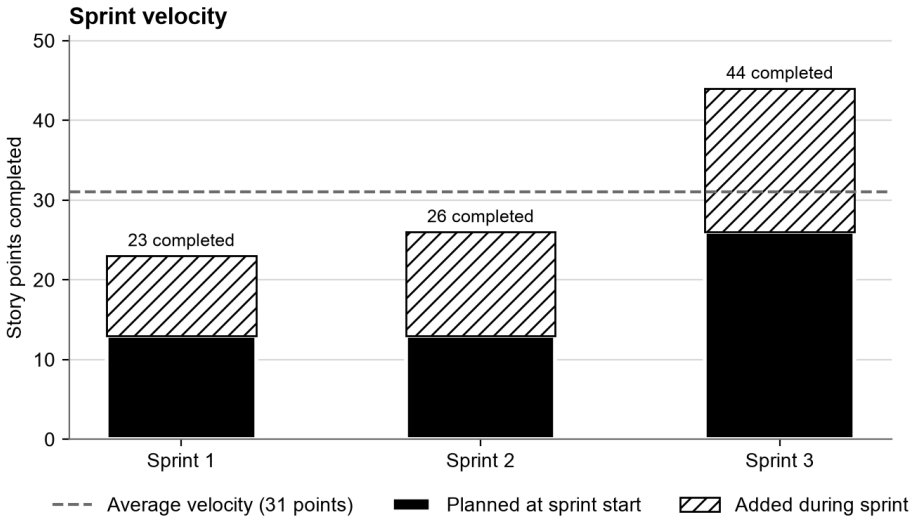


Figure 2. Velocity per sprint

Sprint 1

Deliver the functional core: a customer input form with sample customers, real-time XGBoost prediction with a probability visualiser, input validation, and a clear, light user interface with a risk gauge and churn drivers.

- #1 Interactive Customer Profile Input Form with Presets (5 points)
- #2 Real-time XGBoost Prediction Engine and Probability Visualizer (5 points)
- #3 Robust Input Validation and Boundary Guards (3 points)
- #13 Bright Theme Redesign and One-Click Persona Preset Cards (5 points)
- #14 Real-Time Risk Gauge, Key Driver Breakdown and Boundary Feedback (5 points)

All five stories were completed (23 of 23 points). The product could take a customer profile, validate it, and show a churn probability, risk tier and risk drivers in real time. Pull requests #8 and #9 delivered the core stories, the interface stories were delivered in PR #18, and the 1-minute demonstration video was recorded.

Sprint 2

Complete the product: model evaluation and benchmarking, an automated test suite with continuous integration, a what-if retention simulator, decision threshold tuning, and batch CSV scoring, then freeze the documentation.

- #4 Model Performance Benchmarking and Evaluation (5 points)
- #5 Automated Tests for Validation and Prediction Pipeline (5 points)
- #6 GitHub Actions CI Workflow for Automated Verification (3 points)
- #15 Interactive What-If Retention Strategy Simulator (5 points)

- #16 Decision Threshold Tuning, Confusion Matrix and Batch CSV Scoring (5 points)
- #17 Final Verification, Documentation Freeze and Release (3 points)

All six stories were completed (26 of 26 points). The product gained live evaluation with threshold tuning, batch scoring with a downloadable outreach list, and the what-if simulator. The suite reached 98 automated tests, every CI run passed, and the 2-minute demonstration video was recorded. PR #18 closed issues #15, #16 and #17.

Sprint 3

Compare classical, deep and quantum machine learning on the churn problem and show all three predictions in the application; make experiments reproducible with DVC and MLflow; update and automate every test case and produce the Part 3 test reports.

- S3-1 DVC pipeline and MLflow experiment tracking (5 points)
- S3-2 Code quality refactor and pylint configuration (3 points)
- S3-3 Deep learning (DML) churn model (5 points)
- S3-4 Quantum machine learning (QML) churn model (8 points)
- S3-5 ML vs DML vs QML benchmark and side-by-side predictions (5 points)
- S3-6 Updated test cases and automation of TC-01 to TC-26 (5 points)
- S3-7 Unit, system validation, code coverage and branch coverage reports (3 points)
- S3-8 CI upgrade: lint, coverage gate and report artifacts (3 points)
- S3-9 SRS, SAD and Test Plan v1.1, traceability and deviations (5 points)
- S3-10 README, project report and repository tidy-up (2 points)

All ten stories were completed in code (44 of 44 points) ahead of the sprint window. The three paradigms reach ROC-AUC between 0.829 and 0.846 on the same test set, and the application shows all three predictions for every customer. The test suite grew to 152 tests covering all 26 test cases, with 89.1% statement coverage, 79.4% branch coverage and a pylint score of 9.82/10.

11. Conclusion and Future Work

The project delivers a tested churn prediction system that catches three out of four churners, explains its predictions, and compares classical, deep and quantum machine learning side by side. It was built in three sprints totalling 93 story points, with full requirements, design and test documentation.

- Package the application in a container and serve the model through an API.
- Monitor data drift and schedule retraining.
- Run the quantum classifier on real quantum hardware.